

Programação Orientada a Objetos

Episódio I - Classes e métodos

Rodrigo Cane

SENAI

13 de agosto de 2025

Índice

1 Introdução à POO

2 POO na prática

Introdução à POO

1 Introdução à POO

2 POO na prática

Paradigmas de Programação

- **O que é um Paradigma?**

- É uma filosofia ou um estilo para estruturar algo de maneira padronizada.
- Na programação é o que define a forma como pensamos e resolvemos um problema.

- **Paradigmas Atualmente em Uso**

Paradigmas de Programação

- **O que é um Paradigma?**

- É uma filosofia ou um estilo para estruturar algo de maneira padronizada.
- Na programação é o que define a forma como pensamos e resolvemos um problema.

- **Paradigmas Atualmente em Uso**

- **Declarativo:** Foco no **quê** deve ser feito, não no **como**. O programador descreve o resultado desejado, e o sistema descobre como alcançá-lo.
 - *Exemplos:* SQL.

Paradigmas de Programação

- **O que é um Paradigma?**

- É uma filosofia ou um estilo para estruturar algo de maneira padronizada.
- Na programação é o que define a forma como pensamos e resolvemos um problema.

- **Paradigmas Atualmente em Uso**

- **Declarativo:** Foco no **quê** deve ser feito, não no **como**. O programador descreve o resultado desejado, e o sistema descobre como alcançá-lo.
 - *Exemplos:* SQL.
- **Procedural:** Foco em uma sequência de instruções e funções. Os dados e as funções que os manipulam são separados.
 - *Exemplos:* Python (scripts), C.

Paradigmas de Programação

- **O que é um Paradigma?**

- É uma filosofia ou um estilo para estruturar algo de maneira padronizada.
- Na programação é o que define a forma como pensamos e resolvemos um problema.

- **Paradigmas Atualmente em Uso**

- **Declarativo:** Foco no **quê** deve ser feito, não no **como**. O programador descreve o resultado desejado, e o sistema descobre como alcançá-lo.
 - *Exemplos:* SQL.
- **Procedural:** Foco em uma sequência de instruções e funções. Os dados e as funções que os manipulam são separados.
 - *Exemplos:* Python (scripts), C.
- **Orientada a Objetos (POO):** Foco em modelar o mundo real com **objetos**. Um objeto combina dados (atributos) e o comportamento que opera nesses dados (métodos).
 - *Exemplos:* Python, Java, C#.

Introdução à Programação Orientada a Objetos (POO)

- **Um novo paradigma de programação:**

- Abordagem de modelar o mundo real em software.
- Programas são organizados em torno de dados (objetos), e não em lógica ou funções.

- **Pensamento com 'Objetos':**

- Objetos possuem **atributos** (dados, características).
- Objetos possuem **métodos** (ações, comportamentos).

- **Benefícios:**

- Reutilização de código.
- Facilidade de manutenção.
- Melhor organização e estrutura do software.

Classes e Objetos

- **Classe:**

- É um "molde" ou "planta" para criar objetos.
- Define a estrutura (atributos) e o comportamento (métodos) que todos os objetos desse tipo terão.
- Exemplo: A classe 'ContaCorrente' define que toda conta tem um 'saldo', um 'titular', e os métodos 'depositar()' e 'sacar()'.

- **Objeto:**

- É uma **instância** concreta de uma classe.
- Representa um item específico com seus próprios valores para os atributos definidos na classe.
- Exemplo: Uma conta específica com 'saldo = 1500.00' e 'titular = 'João Silva''.

Encapsulamento e Abstração

● Encapsulamento:

- Combina dados (atributos) e o comportamento (métodos) que opera nesses dados em uma única unidade (a classe).
- **Princípio da Informação Oculta:** Esconde os detalhes internos da implementação de um objeto, permitindo que apenas os métodos designados possam interagir com seus atributos.
- Acesso aos dados é feito de forma controlada.

● Abstração:

- Foca nos aspectos essenciais de um objeto e ignora os detalhes não importantes para o usuário.
- É a capacidade de apresentar a funcionalidade de um objeto de forma simples e clara.
- Exemplo: Ao usar o método 'depositar()', o usuário não precisa saber como o cálculo é feito internamente. Ele apenas "abstrai" o processo.

Classe - Geminado

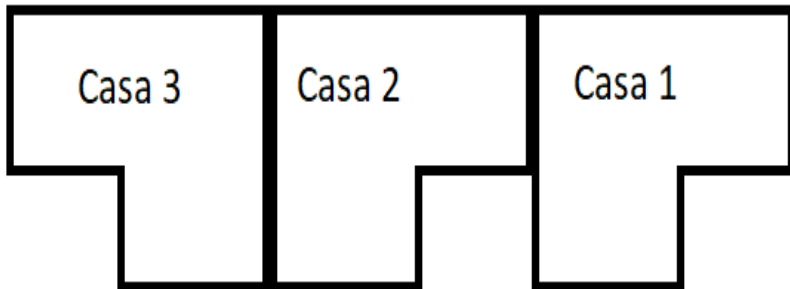
- Muito tempo atrás havia uma casa grande e fuleira. Um belo dia o dono demoliu-a e construiu três geminados no terreno. Eu comprei um desses geminados.

Classe - Geminado

- Muito tempo atrás havia uma casa grande e fuleira. Um belo dia o dono demoliu-a e construiu três geminados no terreno. Eu comprei um desses geminados.
- Um projeto. Três casas "iguais". Exceto a minha, que é meio que "espelhada" em relação as outras duas irmãs.

Classe - Geminado

- Muito tempo atrás havia uma casa grande e fuleira. Um belo dia o dono demoliu-a e construiu três geminados no terreno. Eu comprei um desses geminados.
- Um projeto. Três casas "iguais". Exceto a minha, que é meio que "espelhada" em relação as outras duas irmãs.
- Olhando de cima as casas ficam tipo assim:



Projetando e Construindo

- Em algum momento engenheiro civil, arquiteto e o proprietário bolaram esses geminados.

Projetando e Construindo

- Em algum momento engenheiro civil, arquiteto e o proprietário bolaram esses geminados.
- O projeto original, de uma delas, podemos entender como a classe Geminado.

Projetando e Construindo

- Em algum momento engenheiro civil, arquiteto e o proprietário bolaram esses geminados.
- O projeto original, de uma delas, podemos entender como a classe Geminado.
- Essa classe "deu a luz" a três objetos (que chamamos de instâncias). Foram construídas 3, mas poderiam ser construídas muitas outras.

Projetando e Construindo

- Em algum momento engenheiro civil, arquiteto e o proprietário bolaram esses geminados.
- O projeto original, de uma delas, podemos entender como a classe Geminado.
- Essa classe "deu a luz" a três objetos (que chamamos de instâncias). Foram construídas 3, mas poderiam ser construídas muitas outras.
- **Propriedades:** `numero_casa`, `numero_quartos`, `numero_banheiros`, `cor_externa`, `proprietarios[]`, `moradores[]`, `esta_ocupada`

Projetando e Construindo

- Em algum momento engenheiro civil, arquiteto e o proprietário bolaram esses geminados.
- O projeto original, de uma delas, podemos entender como a classe Geminado.
- Essa classe "deu a luz" a três objetos (que chamamos de instâncias). Foram construídas 3, mas poderiam ser construídas muitas outras.
- **Propriedades:** numero_casa, numero_quartos, numero_banheiros, cor_externa, proprietarios[], moradores[], esta_ocupada
- **Métodos:** desarmar_disjuntor(), abrir_portao(), verificar_ocupacao()

Sobre as props e métodos

- As propriedades nascem com um valor padrão, mas podem ser alteradas ao longo do tempo pelos métodos.

Sobre as props e métodos

- As propriedades nascem com um valor padrão, mas podem ser alteradas ao longo do tempo pelos métodos.
- Veja que deixei “ocupação” como prop e como método. Isso porque é uma questão de estilo de programação e convenções. Se uma prop é o resultado de um cálculo (em cima de outras props ou não), então o melhor é torná-la um método.

Sobre as props e métodos

- As propriedades nascem com um valor padrão, mas podem ser alteradas ao longo do tempo pelos métodos.
- Veja que deixei “ocupação” como prop e como método. Isso porque é uma questão de estilo de programação e convenções. Se uma prop é o resultado de um cálculo (em cima de outras props ou não), então o melhor é torná-la um método.
- Mas e o tal do “espelhamento”? Faz sentido ter um método `espelhar()`?

Sobre as props e métodos

- As propriedades nascem com um valor padrão, mas podem ser alteradas ao longo do tempo pelos métodos.
- Veja que deixei “ocupação” como prop e como método. Isso porque é uma questão de estilo de programação e convenções. Se uma prop é o resultado de um cálculo (em cima de outras props ou não), então o melhor é torná-la um método.
- Mas e o tal do “espelhamento”? Faz sentido ter um método `espelhar()`?
- Não po! Dá pra mudar muita coisa numa casa, mas não dá pra inventar ela horizontalmente.

Sobre as props e métodos

- As propriedades nascem com um valor padrão, mas podem ser alteradas ao longo do tempo pelos métodos.
- Veja que deixei “ocupação” como prop e como método. Isso porque é uma questão de estilo de programação e convenções. Se uma prop é o resultado de um cálculo (em cima de outras props ou não), então o melhor é torná-la um método.
- Mas e o tal do “espelhamento”? Faz sentido ter um método `espelhar()`?
- Não po! Dá pra mudar muita coisa numa casa, mas não dá pra inventar ela horizontalmente.
- Esse tipo de “característica” normalmente seria passado no construtor (um tipo especial de método que... CONSTRÓI um objeto) e então nunca mais poderia ser mudada.

Classe - Carro

- **Propriedades:** é_automatico, motor, numero_passageiros, cor, modelo, motor_ligado, farol_aceso,

Classe - Carro

- **Propriedades:** `é_automatico`, `motor`, `numero_passageiros`, `cor`, `modelo`, `motor_ligado`, `farol_aceso`,
- **Métodos:** `ligar_carro()`, `acender_farol()`, `engatar_marcha()`, `acelerar()`, `frear()`, `debrear()`, `buzinar()`

Classe - Carro

- **Propriedades:** `é_automatico`, `motor`, `numero_passageiros`, `cor`, `modelo`, `motor_ligado`, `farol_aceso`,
- **Métodos:** `ligar_carro()`, `acender_farol()`, `engatar_marcha()`, `acelerar()`, `frear()`, `debrear()`, `buzinar()`
- Aqui de novo temos propriedades “fixas” (“não dá” pra “automatizar” o carro ou trocar o motor dele depois de pronto) e propriedades “variáveis” (`cor`, `motor ligado` etc).

Mais sobre props e métodos

- Ou seja, características permanentes e “estados” são todos representados como PROP.

Mais sobre props e métodos

- Ou seja, características permanentes e “estados” são todos representados como PROP.
- Métodos são as ações, que geralmente influenciam as props do próprio objeto, mas podem afetar o mundo exterior também.

Mais sobre props e métodos

- Ou seja, características permanentes e “estados” são todos representados como PROP.
- Métodos são as ações, que geralmente influenciam as props do próprio objeto, mas podem afetar o mundo exterior também.
- Usei a palavra ESTADO ali porque é bem comum pensarmos em termos de estado. Quando ligo o carro ele vai pro estado “ligado”.

Mais sobre props e métodos

- Ou seja, características permanentes e “estados” são todos representados como PROP.
- Métodos são as ações, que geralmente influenciam as props do próprio objeto, mas podem afetar o mundo exterior também.
- Usei a palavra ESTADO ali porque é bem comum pensarmos em termos de estado. Quando ligo o carro ele vai pro estado “ligado”.
- Faz sentido eu acelerar um carro que está no estado desligado?

Mais sobre props e métodos

- Ou seja, características permanentes e “estados” são todos representados como PROP.
- Métodos são as ações, que geralmente influenciam as props do próprio objeto, mas podem afetar o mundo exterior também.
- Usei a palavra ESTADO ali porque é bem comum pensarmos em termos de estado. Quando ligo o carro ele vai pro estado “ligado”.
- Faz sentido eu acelerar um carro que está no estado desligado?
- Frear até faz sentido (talvez o carro esteja num morro).

Mais sobre props e métodos

- Ou seja, características permanentes e “estados” são todos representados como PROP.
- Métodos são as ações, que geralmente influenciam as props do próprio objeto, mas podem afetar o mundo exterior também.
- Usei a palavra ESTADO ali porque é bem comum pensarmos em termos de estado. Quando ligo o carro ele vai pro estado “ligado”.
- Faz sentido eu acelerar um carro que está no estado desligado?
- Frear até faz sentido (talvez o carro esteja num morro).

CHEGA DE BRINCADEIRAS! Bora pra um exemplo no Python.

POO na prática

1 Introdução à POO

2 POO na prática

Definição

Faremos um sisteminha que controla um caixa eletrônico. A primeira coisa é colocar o cartão. Vamos simular isso digitando 3 números. Aí então vem o menu principal, que fica em loop enquanto o usuário quiser fazer mais operações. As operações são:

- 1 - Saldo
- 2 - Depósito
- 3 - Saque
- 4 - Sair

Não temos BD ainda. Vamos deixar em variáveis pra simular.

Vou lendo aqui e mostrando no VS o código já pré-programado, tipo a Ana Maria Braga mostrando uma receita de bolo, mas ela já tem uns 3 ou 4 bolos em estágios diferentes da preparação.

v1 - 100% Procedural

- **Desafio:** Começaremos com um único cliente na “base” do caixa: Rodrigo Rodrigues, cartão 123, conta 0171-9 e saldo inicial de R\$5000.

v1 - 100% Procedural

- **Desafio:** Começaremos com um único cliente na “base” do caixa: Rodrigo Rodrigues, cartão 123, conta 0171-9 e saldo inicial de R\$5000.
- **Resultado parcial:** Versão inicial, sequencial e o que acredito que vocês fariam inicialmente.

v1 - 100% Procedural

- **Desafio:** Começaremos com um único cliente na “base” do caixa: Rodrigo Rodrigues, cartão 123, conta 0171-9 e saldo inicial de R\$5000.
- **Resultado parcial:** Versão inicial, sequencial e o que acredito que vocês fariam inicialmente.
- E tá tudo bem.

v1 - 100% Procedural

- **Desafio:** Começaremos com um único cliente na “base” do caixa: Rodrigo Rodrigues, cartão 123, conta 0171-9 e saldo inicial de R\$5000.
- **Resultado parcial:** Versão inicial, sequencial e o que acredito que vocês fariam inicialmente.
- E tá tudo bem.
- E é sobre isso.

v1 - 100% Procedural

- **Desafio:** Começaremos com um único cliente na “base” do caixa: Rodrigo Rodrigues, cartão 123, conta 0171-9 e saldo inicial de R\$5000.
- **Resultado parcial:** Versão inicial, sequencial e o que acredito que vocês fariam inicialmente.
- E tá tudo bem.
- E é sobre isso.
- **Problemas dessa versão:** mas tá ruim demais né? Tudo espalhado, difícil de evoluir.

v2 - Várias contas

- **Desafio:** Caixa pra uma conta só não dá né? Vamos flexibilizar pra termos várias contas possíveis.

v2 - Várias contas

- **Desafio:** Caixa pra uma conta só não dá né? Vamos flexibilizar pra termos várias contas possíveis.
- **Resultado parcial:** Dicionário fez um cosplay de BD.

v2 - Várias contas

- **Desafio:** Caixa pra uma conta só não dá né? Vamos flexibilizar pra termos várias contas possíveis.
- **Resultado parcial:** Dicionário fez um cosplay de BD.
- **Problemas dessa versão:** O dicionário já dá a dica de características da conta a serem “agrupadas”. Tá esquisito isso estar por todo canto do código, né não? E outra... se eu tiver que mudar a forma como o saldo é exibido, por exemplo, em quantos lugares eu teria que mexer?

v3 - Uma tímida classe nasce

- **Desafio:** Centralizar a lógica da Conta. Reaproveitar código onde possível.

v3 - Uma tímida classe nasce

- **Desafio:** Centralizar a lógica da Conta. Reaproveitar código onde possível.
- **Resultado parcial:** Nasce a classe ContaCorrente. Agora `mostrar_saldo` é um método.

v3 - Uma tímida classe nasce

- **Desafio:** Centralizar a lógica da Conta. Reaproveitar código onde possível.
- **Resultado parcial:** Nasce a classe ContaCorrente. Agora `mostrar_saldo` é um método.
- **Problemas dessa versão:** usamos o `depositar()` só porque quisemos. O campo `saldo` tava ali público, pra quem quisesse alterar (baixo encapsulamento).

v4 - Encapsulamento 101

- **Desafio:** Encapsular um pouco pra obrigar o usuário a chamar depositar() e sacar() se ele quiser alterar o saldo da conta.

v4 - Encapsulamento 101

- **Desafio:** Encapsular um pouco pra obrigar o usuário a chamar depositar() e sacar() se ele quiser alterar o saldo da conta.
- **Resultado parcial:** Adicionando o @property e alterando o nome das props pra “__prop” deixamos público o o get e ocultamos o set.

v4 - Encapsulamento 101

- **Desafio:** Encapsular um pouco pra obrigar o usuário a chamar depositar() e sacar() se ele quiser alterar o saldo da conta.
- **Resultado parcial:** Adicionando o @property e alterando o nome das props pra “__prop” deixamos público o o get e ocultamos o set.
- **Problemas dessa versão:** Já tá legalziN até, mas ainda tá meio misto de POO e Procedural né?

v5 - Acabou o dinheiro infinito

- **Desafio:** Estamos simulando o software de UM caixa eletrônico só, não de toda a rede bancária. Então faz sentido ele ter um saldo também né?

v5 - Acabou o dinheiro infinito

- **Desafio:** Estamos simulando o software de UM caixa eletrônico só, não de toda a rede bancária. Então faz sentido ele ter um saldo também né?
- **Resultado parcial:** Checamos se o caixa tem saldo pro saque, com o cuidado de nunca expor o valor exato que o caixa tem.

v5 - Acabou o dinheiro infinito

- **Desafio:** Estamos simulando o software de UM caixa eletrônico só, não de toda a rede bancária. Então faz sentido ele ter um saldo também né?
- **Resultado parcial:** Checamos se o caixa tem saldo pro saque, com o cuidado de nunca expor o valor exato que o caixa tem.
- **Problemas dessa versão:** Agora tá visivelmente desconfortável essa mistura né? Não sou só eu...

v6 - Finalmente POO

- **Desafio:** Orientar a objeto memo. Nada desse loop flutuando no espaço.

v6 - Finalmente POO

- **Desafio:** Orientar a objeto memo. Nada desse loop flutuando no espaço.
- **Resultado parcial:** Classe CaixaEletronico criada, concentrando a lógica do caixa em si. Também aproveitamos pra separar os fontes em arquivos individuais que podem ser importados.

v6 - Finalmente POO

- **Desafio:** Orientar a objeto memo. Nada desse loop flutuando no espaço.
- **Resultado parcial:** Classe CaixaEletronico criada, concentrando a lógica do caixa em si. Também aproveitamos pra separar os fontes em arquivos individuais que podem ser importados.
- **Problemas dessa versão:** Essa versão ainda peca na separação das responsabilidades, que acaba causando algumas checagens duplicadas também. Mas deixemos isso pra outra aula!

v7 - Multi DEV

- **Desafio:** será que o CaixaEletronico.py que você fez roda com o ContaCorrente.py do amiguinho? E será que você consegue sanar os deslizes que citei anteriormente?

Perguntas

Perguntas?